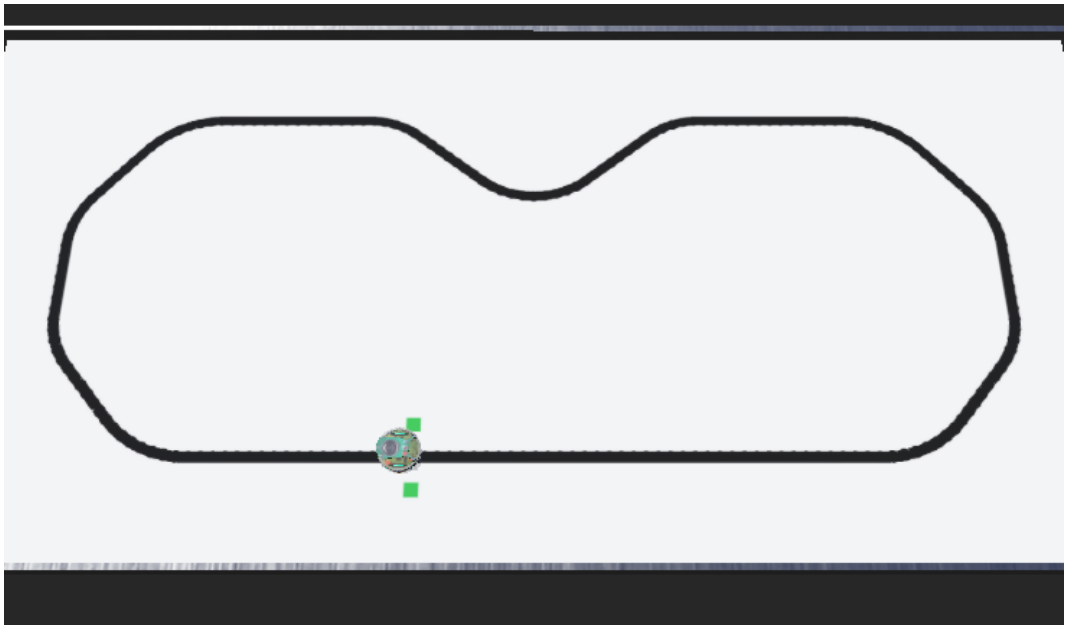
Zadanie 1: Line follower



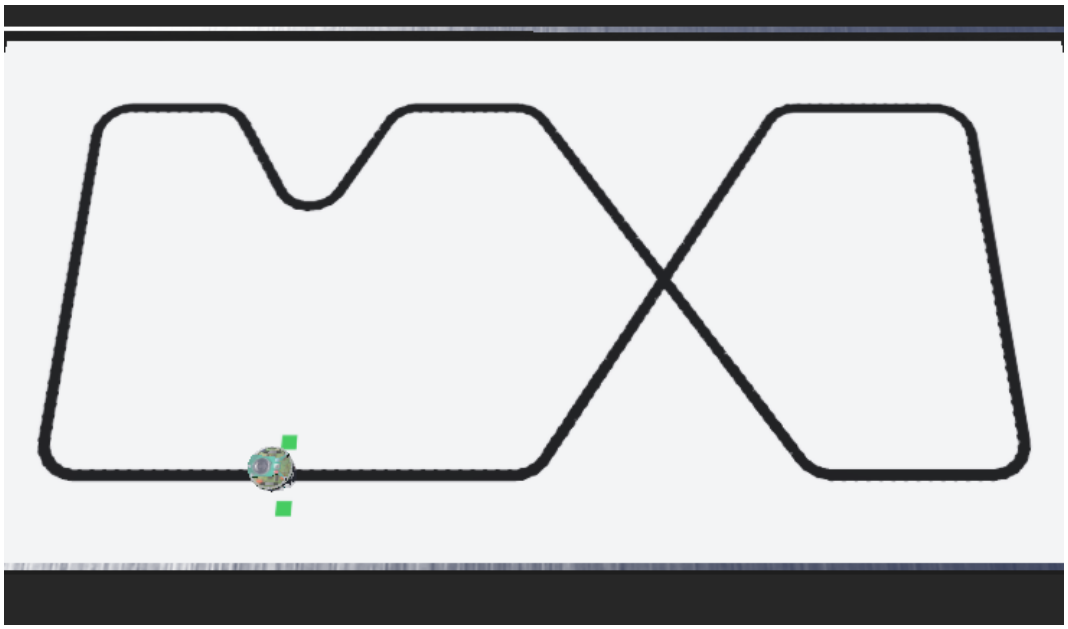
Opis zadania

Robot e-puck stoi na torze z czarną linią. Napisz program, który sprawi, że robot przejedzie **jedno pełne okrążenie** wzdłuż linii, zaczynając i kończąc przy zielonych znacznikach startu. Liczy się czas.

Przygotowaliśmy dwa tory:

- **Tor 1 (łatwy)**, plik `worlds/track_1.wbt`: ok. 4,2 m, łagodne łuki.
- **Tor 2 (trudny)**, plik `worlds/track_2.wbt`: ok. 5,5 m, ostre zakręty, zygzak w kształcie litery V i skrzyżowanie, na którym trzeba jechać prosto.

Ten sam program powinien przejechać oba tory bez zmian.



Struktura folderu

```
01_line_follower/
├── worlds/
│   ├── track_1.wbt                      tor 1
│   ├── track_2.wbt                      tor 2
│   └── textures/                        obrazy torów
└── controllers/
    ├── line_follower/line_follower.py   TWÓJ PROGRAM
    └── judge/                           sędzia (nie modyfikuj)
```

Robot

Robot e-puck ma dwa koła napędzane silnikami i trzy czujniki podłoża z przodu.

- `left wheel motor`, `right wheel motor`: silniki kół. Program ustawia prędkość obrotową, maksymalnie 6,28 rad/s (ok. 13 cm/s).
- `gs0` (lewy), `gs1` (środkowy), `gs2` (prawy): czujniki podłoża, rozstawione co 1 cm, ok. 3 cm przed osią kół. Zwracają ok. **300 na czarnej linii** i ok. **750 na białym podłożu**. Wartości pośrednie pojawiają się tylko na samej krawędzi linii, więc w praktyce każdy czujnik mówi „linia” albo „brak linii”.
- Linia ma szerokość 2 cm. Gdy robot jedzie dokładnie po środku, środkowy czujnik widzi linię, a boczne białe tło.

Szkielet programu w `line_follower.py` konfiguruje już silniki i czujniki. Na razie robot jedzie prosto. Twoje zadanie to uzupełnić część oznaczoną `TODO`.

Zasady i ocena

Sędzia (robot „judge”) śledzi pozycję e-pucka i wypisuje wyniki w konsoli.

- Okrążenie liczy się, gdy robot przejedzie cały tor we właściwym kierunku i wróci na start. Skróty nie są zaliczane.
- Jeśli robot oddali się od linii (więcej niż 6 cm) na dłużej niż 3 s, próba się kończy.
- Limit czasu: 180 s na okrążenie.
- Wynik to czas okrążenia. Najpierw liczy się liczba przejechanych torów, potem suma czasów.

Poziomy:

- **podstawowy**: przejazd toru 1,
- **pełny**: przejazd obu torów,
- **dla chętnych**: jak najkrótsze czasy. Dla porównania nasze rozwiązanie wzorcowe przejeżdża tor 1 w ok. 47 s, a tor 2 w ok. 60 s.

Wskazówki

- Zacznij od wypisania wartości czujników w konsoli (linia z `print` w szkielecie jest zakomentowana) i obejrzyj, jak zmieniają się w zakrętach.
- Najprostsze podejście: jeśli linię widzi lewy czujnik, skręć w lewo, jeśli prawy, skręć w prawo. Lepsze: regulator proporcjonalny (P) albo PD, w którym siła skrętu zależy od błędu.
- Zastanów się, co robot ma zrobić, gdy żaden czujnik nie widzi linii (np. po zbyt szybkim zakręcie) i gdy wszystkie widzą linię (skrzyżowanie).
- Szybkość to kompromis: zwolnij w zakrętach, przyspiesz na prostych.

Co oddać

- plik sterownika (`.py`) z Twoim rozwiązaniem,
- krótkie nagranie ekranu (do 2 minut) pokazujące przejazd/działanie robota i wynik sędziego w konsoli,
- kilka zdań opisu: jak działa Twoje rozwiązanie, co sprawiało trudność i co byś poprawił(a), mając więcej czasu.

Rozwiązanie wyślij w sposób podany przez organizatorów rekrutacji. Możesz korzystać z dokumentacji, internetu i narzędzi AI, ale na rozmowie poprosimy Cię o wyjaśnienie swojego kodu, więc upewnij się, że rozumiesz każdą jego linijkę.

Nie modyfikuj plików sędziego ani świata (poza eksperymentami na własny użytek). Przy ocenie uruchomimy Twój sterownik w oryginalnych światach.

Webots: co to jest i jak zacząć

Czym jest Webots

Webots to darmowy, otwartoźródłowy symulator robotów rozwijany przez firmę Cyberbotics. Pozwala zbudować wirtualny świat z robotami, czujnikami i obiektami, a potem sterować robotami programami (tzw. kontrolerami) napisanymi m.in. w Pythonie, C lub C++. Symulacja uwzględnia fizykę: tarcie, masy, zderzenia i szum czujników, więc program, który działa w Webots, zachowuje się podobnie jak na prawdziwym robocie. Dzięki temu możesz uczyć się robotyki bez sprzętu.

Instalacja

1. Pobierz **Webots R2025a** ze strony https://cyberbotics.com (przycisk Download) albo z https://github.com/cyberbotics/webots/releases. Zadania przygotowano i przetestowano w tej wersji.
2. Zainstaluj program jak zwykłą aplikację (Windows: instalator `.exe`, macOS: plik `.dmg`, Linux: pakiet `.deb` lub `snap`).
3. Zainstaluj **Python 3** (https://www.python.org). Na Windows zaznacz w instalatorze opcję „Add Python to PATH”.
4. Jeśli Webots nie znajdzie Pythona sam, wskaż go w *Tools* → *Preferences* → *General* → *Python command* (np. `python` albo pełna ścieżka do `python.exe`).

Pierwsze uruchomienie zadania

1. Pobierz to repozytorium (`git clone` albo *Code* → *Download ZIP*) i rozpakuj je w zwykłym folderze, np. `C:\robotyka\`. **Nie** umieszczaj go w folderze instalacji Webots (`C:\Program Files\Webots`) ani w folderze synchronizowanym z OneDrive.
2. W Webots wybierz *File* → *Open World* i otwórz plik `.wbt` z folderu `worlds` wybranego zadania.
3. Przy pierwszym otwarciu Webots pobiera modele robotów z internetu, więc potrzebne jest połączenie z siecią. Może to potrwać chwilę.
4. Symulację uruchamiasz przyciskiem play (▶), zatrzymujesz przyciskiem pauzy, a przycisk reset (strzałka zawracająca) przywraca stan początkowy.
5. Komunikaty programów, w tym wynik od sędziego, pojawiają się w konsoli na dole okna.

Jak edytować program robota

Kod sterownika znajduje się w folderze `controllers/<nazwa>/<nazwa>.py`. Możesz go edytować we wbudowanym edytorze Webots (prawy klik na robota → *Edit controller*) albo w dowolnym edytorze, np. VS Code. Po zapisaniu zmian zresetuj symulację, żeby program uruchomił się od nowa.

Przydatne źródła:

- dokumentacja Webots: https://cyberbotics.com/doc/guide/index
- opis funkcji (API) dla Pythona: https://cyberbotics.com/doc/reference/index (przełącz język przykładów na Python)
- tutoriale dla początkujących: https://cyberbotics.com/doc/guide/tutorials

Typowe problemy

- **„Skipped unknown … node or PROTO”** lub **„changed by fallback mechanism”**: Webots nie może pobrać modeli. Sprawdź połączenie z internetem i to, czy projekt nie leży w folderze instalacji Webots.
- **Sterownik nie startuje, błąd Pythona**: sprawdź ustawienie *Python command* w preferencjach.
- **Zmiany w kodzie nie działają**: zapisz plik i zresetuj symulację.
- **Symulacja działa wolno**: wyłącz cienie i antyaliasing w *Tools* → *Preferences* → *OpenGL* albo użyj trybu szybkiego (przycisk z podwójną strzałką, *Fast*).